



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3

із дисципліни «Технології розробки програмного забезпечення»

Тема: «Діаграма розгортання. Діаграма компонентів. Діаграма взаємодій та послідовностей»

Виконав:
Студент групи ІА-24
Зелінський І.О.

Перевірив:
Мякий М. Ю.

Зміст

Тема.....	3
Мета.....	3
Обрана тема	3
Опис трьох прецедентів.....	3
1. Адміністратор керує піснями (видаляє/додає).....	3
2. Користувач переглядає історію прослуховувань	4
3. Користувач створює свій плейлист.....	4
Діаграма прецедентів.....	5
Діаграма класів.....	6
Опис моделей.....	7
Зв'язки між моделями.....	8
Структура БД	9
Висновок.....	9

Тема: Діаграма розгортання. Діаграма компонентів. Діаграма взаємодій та послідовностей

Мета: Навчитися створювати діаграми розгортання, компонентів та послідовностей у нотації UML для моделювання програмних систем

Завдання:

1. Ознайомитися з короткими теоретичними відомостями.
2. Розробити діаграму розгортання для проектованої системи.
3. Розробити діаграму компонентів для проектованої системи.
4. Розробити діаграму послідовностей для проектованої системи.
5. Скласти звіт про виконану роботу.

Теоретичні відомості

Діаграма компонентів UML відображає систему, розбиту на окремі модулі. Залежно від способу поділу на модулі розрізняють три типи діаграм компонентів:

1. Логічні — часто використовуються, оскільки система уявляється як набір самостійних, незалежних модулів (компонентів), що взаємодіють між собою.
2. Фізичні — відображають реальну структуру компонентів у системі.
3. Виконувані — показують компоненти, які безпосередньо запускаються.

Діаграми розгортання представляють фізичне розташування системи, показуючи, на якому обладнанні запускаються різні елементи програмного забезпечення. Основними елементами цієї діаграми є вузли, пов'язані інформаційними каналами. Вузол (node) може містити програмне забезпечення. Існують два типи вузлів:

1. Пристрої (device) — це фізичне обладнання, наприклад, комп'ютер або інший пристрій, що пов'язаний із системою.
2. Середовище виконання (execution environment) — це програмне забезпечення, яке може містити інше ПЗ.

Діаграма послідовностей демонструє, як об'єкти системи взаємодіють між собою в хронологічному порядку для виконання певних операцій. Вона фокусується на повідомленнях, що передаються між компонентами, та їх порядку. Це корисний інструмент для моделювання динамічних аспектів системи, таких як процес реєстрації користувача або збереження даних. Діаграма допомагає зрозуміти взаємодію між

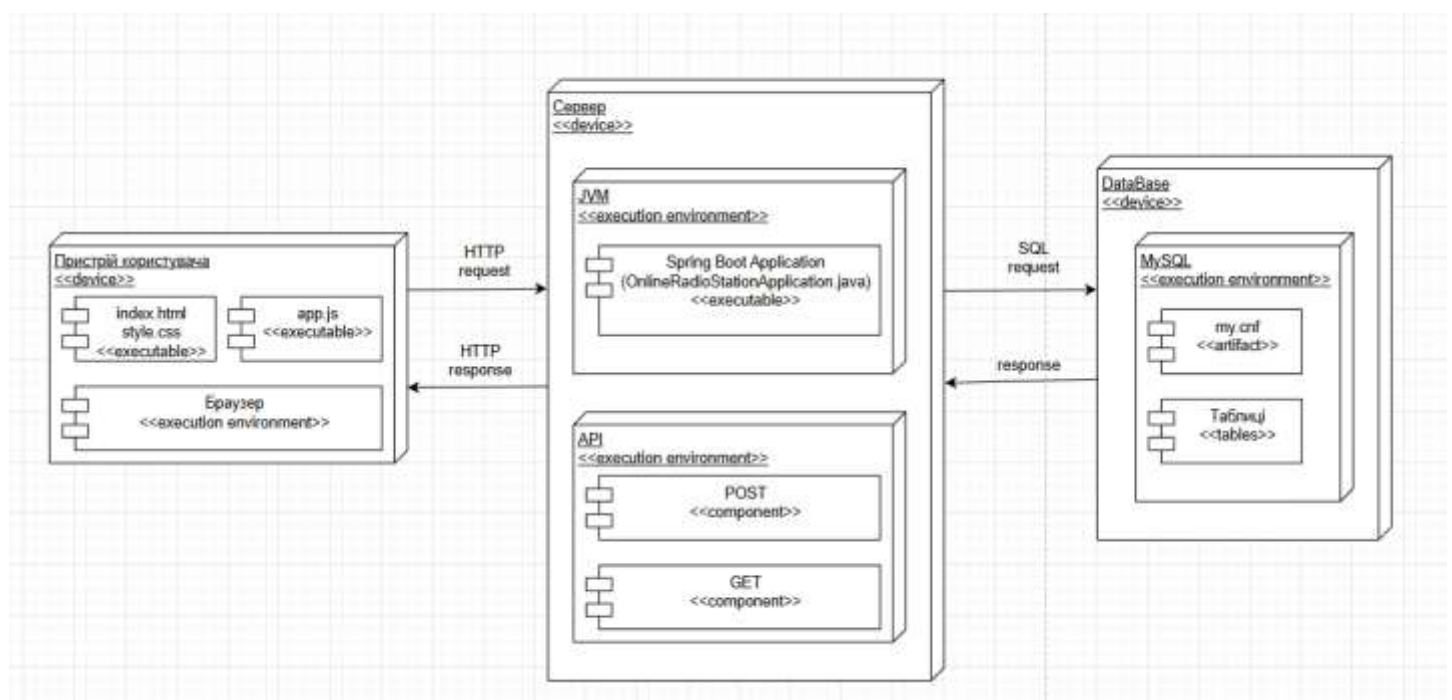
клієнтом, сервером та базою даних, забезпечуючи чітке розуміння порядку виконання операцій у реальному часі.

Обрана тема: Online radio station (iterator, adapter, factory method, facade, visitor, client-server)

Додаток повинен служити сервером для радіостанції з можливістю мовлення на радіостанцію (64, 92, 128, 196, 224 kb/s) в потоковому режимі; вести облік підключених користувачів і статистику відвідувань і прослуховувань; налаштувати папки з піснями і можливість вести списки програвання або playlists (не відтворювати всі пісні).

Хід роботи

Діаграма розгортання для проектованої системи



Пристрій користувача

Тип: device

Компоненти:

`index.html`, `style.css` — статичні файли, HTML-розмітка та CSS-стилі для фронтенду.

`app.js` — Frontend JavaScript-код, що забезпечує взаємодію користувача з інтерфейсом та комунікацію з сервером.

Браузер — середовище виконання, через яке користувач надсилає HTTP-запити до сервера і отримує HTTP-відповіді.

Взаємодія: Користувач взаємодіє з інтерфейсом в браузері. HTTP-запити надсилаються на сервер, і HTTP-відповіді містять дані, потрібні для роботи користувацького інтерфейсу.

Сервер

Тип: device

Компоненти:

JVM — середовище виконання Java, в якому працює основний додаток.

Spring Boot Application (OnlineRadioStationApplication.java) — основна програма на Spring Boot, що забезпечує бізнес-логіку системи.

API

POST — компонент, що обробляє запити на створення даних (наприклад, створення нового плейлиста або додавання пісні).

GET — компонент, що обробляє запити на отримання даних (наприклад, отримання списку пісень або статистики прослуховування).

Взаємодія: Приймає HTTP-запити від користувача через API та повертає HTTP-відповіді. Взаємодіє з базою даних через SQL-запити для зберігання та отримання інформації.

База даних (DataBase)

Тип: device

Компоненти:

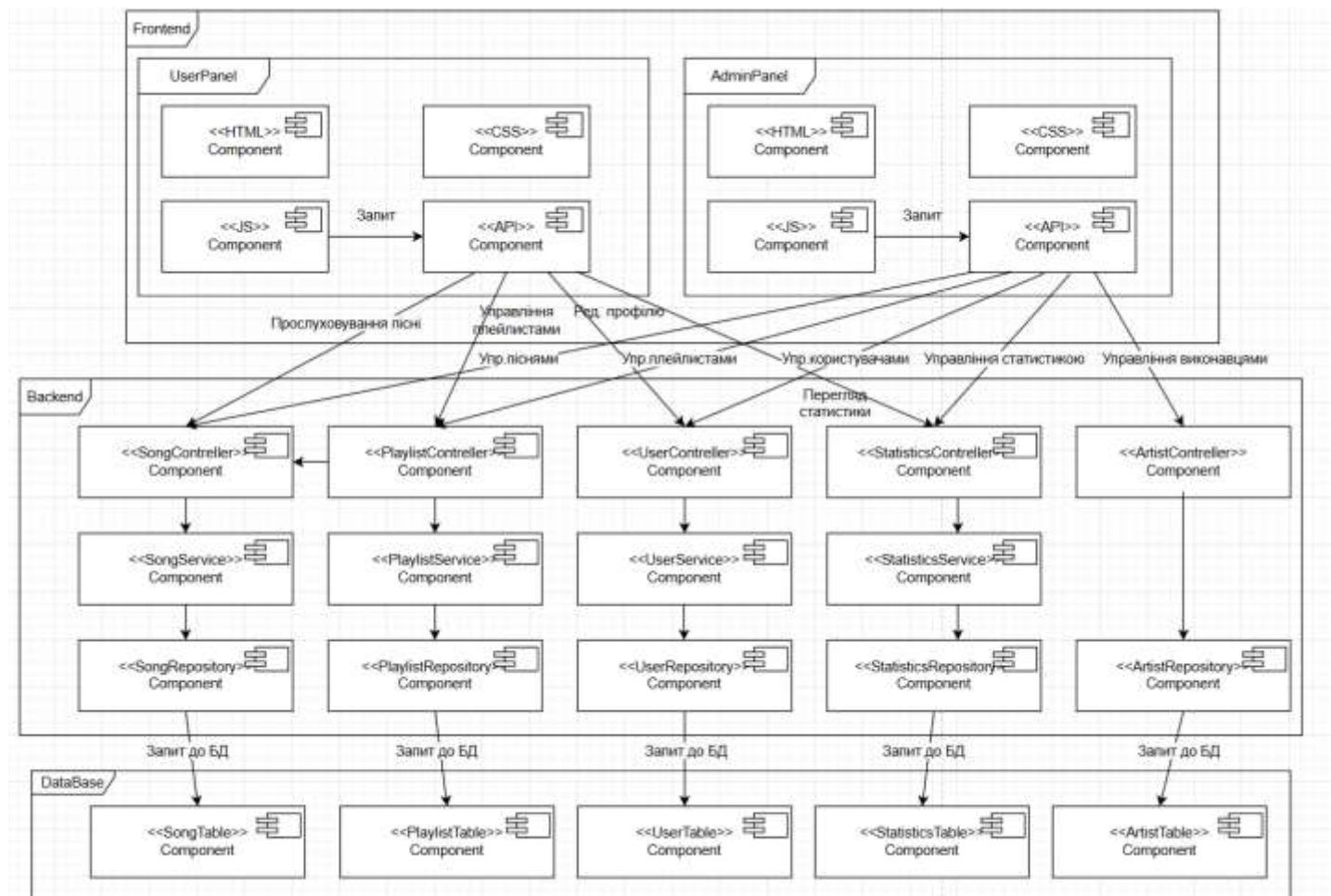
MySQL — середовище виконання бази даних MySQL.

my.cnf — конфігураційний файл MySQL, який зберігає налаштування бази даних.

Таблиці — структури для зберігання інформації про користувачів, пісні, плейлисти та інші дані, пов'язані із системою.

Взаємодія: Приймає SQL-запити від сервера для зберігання, оновлення та отримання даних. Повертає відповідь на сервер після виконання запитів.

Діаграма компонентів



1. Frontend

UserPanel — інтерфейс користувача для доступу до функцій онлайн-радіо.

- HTML — компонент для структури сторінки та елементів інтерфейсу.
- CSS — компонент для стилізації інтерфейсу.
- JS — JavaScript-компонент, що забезпечує взаємодію та динамічні елементи на стороні клієнта.
- API — компонент, який надсилає запити до серверної частини через API для отримання та відправлення даних.

AdminPanel — інтерфейс адміністратора для управління контентом і статистикою.

- HTML — компонент для відображення адміністративного інтерфейсу.
- CSS — компонент стилів для оформлення панелі адміністратора.
- JS — JavaScript-компонент, що відповідає за функціонал адміністративної панелі.
- API — компонент для надсилання запитів до серверної частини, аналогічний до користувацької панелі, але з розширеними правами доступу.

- Взаємодія між компонентами Frontend і Backend:

- UserPanel і AdminPanel надсилають запити на сервер через API для таких операцій, як прослуховування пісень, управління плейлистами, редагування профілю, перегляд статистики, управління виконавцями та користувачами.

2. Backend

- Компоненти серверної частини розділені на Controller, Service, і Repository рівні:

- Controller — приймає HTTP-запити від API та передає їх на обробку в сервісний рівень.

- Service — обробляє бізнес-логіку, а також взаємодіє з рівнем Repository для отримання або збереження даних.

- Repository — відповідає за доступ до бази даних (через запити до таблиць).

- Компоненти Backend:

- SongController, SongService, SongRepository — відповідають за обробку запитів, пов'язаних з піснями, включаючи додавання, видалення і перегляд пісень.

- PlaylistController, PlaylistService, PlaylistRepository — забезпечують управління плейлистами, включаючи створення, редагування і видалення плейлистів.

- UserController, UserService, UserRepository — обробляють операції, пов'язані з користувачами, зокрема управління профілями та реєстрацією.

- StatisticsController, StatisticsService, StatisticsRepository — відповідають за збір та обробку статистичних даних, таких як кількість прослуховувань та активність користувачів.

- ArtistController, ArtistRepository — компоненти для управління інформацією про виконавців, включаючи додавання та редагування даних про артистів.

3. Database

- SongTable — таблиця в базі даних для зберігання інформації про пісні.

- PlaylistTable — таблиця для зберігання інформації про плейлисти.

- UserTable — таблиця для зберігання даних користувачів.

- StatisticsTable — таблиця для зберігання статистичних даних про активність користувачів і прослуховування пісень.

- ArtistTable — таблиця для зберігання даних про виконавців.

- Запити до БД:

- Controller-рівень надсилає запити до Repository-рівня, який передає запити до відповідних таблиць у базі даних для створення, читання, оновлення або видалення записів.

Діаграма послідовностей

